

## An iOS SQLite Database Example Application

We are going to design an application to store an address book, that is contact information (names, addresses and telephone numbers) in a SQLite database. In addition to data storage, a feature will also be implemented to allow the user to search the database for the address and phone number of a specified contact name. Some knowledge of SQL and SQLite database concepts is assumed.

1. Create a new Xcode **Single View Application** project. Call it something like "**Database**" or another title of your choice. Use the Storyboard support and enable Automatic Reference Counting (ARC).
2. Once the project has been created, the next step is to configure the project to include the SQLite dynamic library (libsqlite3.dylib) during the link phase of the build process. Failure to include this library will result in build errors.

- To add this library, select the target entry in the Xcode project navigator (the top entry with the product name) to display the summary information panel.
- Select the **Build Phases tab** to display the build information. The **Link Binary with Libraries** section lists the libraries and frameworks already included in the project. **To add another library or framework click on the '+' button to display the full list. Search and select libsqlite3.dylib. Click "Add."**

3. Let's import sqlite3.h and declaring the Database Reference

Before we can create a database we need to declare a variable pointer to a structure of type sqlite3 that will act as the reference to our database. Since we will be working with the database in the view controller for our application the best place to declare this variable is in the **ViewController.h** file.

Since we also need to import the sqlite3.h header file into any files where we make use of SQLite this is also an ideal place to include the file. Now is also a good time to declare an NSString property in which to store the path to the database. Within the main Xcode project navigator, select the **ViewController.h** file and modify it as follows:

Add to the top of ViewController.h:

```
#import <sqlite3.h>
```

Add the following two properties to this same file:

```
@property (strong, nonatomic) NSString *databasePath;
```

```
@property (nonatomic) sqlite3 *contactDB;
```

The finished file now looks like this:

```
#import <UIKit/UIKit.h>
```

```
#import <sqlite3.h>
```

```
@interface ViewController : UIViewController
```

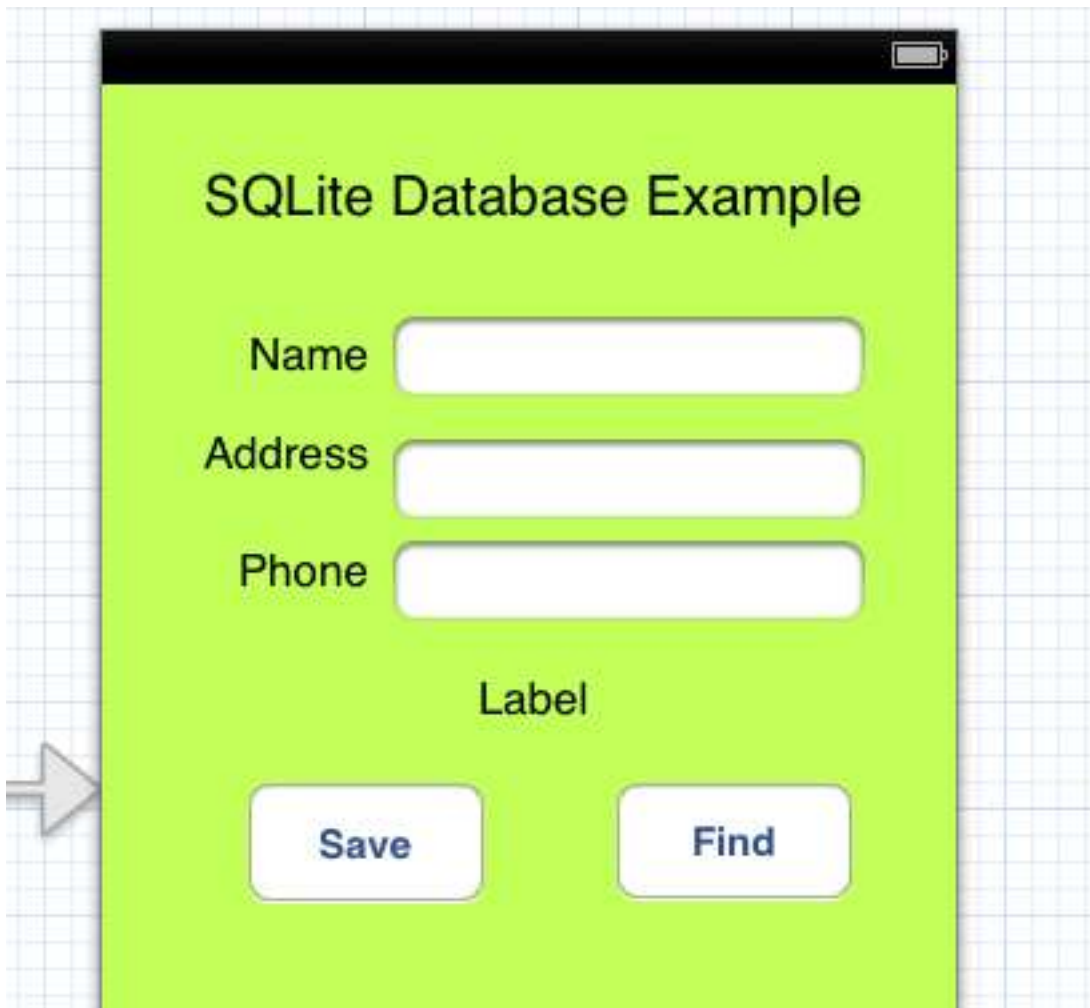
```
@property (strong, nonatomic) NSString *databasePath;
```

```
@property (nonatomic) sqlite3 *contactDB;
```

```
@end
```

4. Now you need to design the user interface.

Select the MainStoryboard.storyboard file to edit the user interface and drag and drop components from the Object library (View -> Utilities -> Object Library) onto the view canvas and edit properties so that the layout appears as shown below:



The user interface for an iPhone SQLite Database example

Note: Before proceeding, stretch the status label (located above the two buttons) so that it covers most of the width of the view. Finally, edit the label and remove the word "Label" so that it is blank.

5. Next you will wire the components to ViewController.h.

Select the top most text field object in the view canvas, display the Assistant Editor panel (tuxedo icon) and verify that the editor is displaying the contents of the ViewController.h file.

- Ctrl-click on the text field object and drag it to a position just below the @interface line. Release the line and in the resulting

connection dialog establish an outlet connection named **theName**.

- Repeat the above steps to establish outlet connections for the remaining text fields and the label object to properties named **theAddress**, **thePhone** and **theStatus**.
- Ctrl-click on the Save button object and drag the line to the area immediately beneath the newly created outlet in the Assistant Editor panel. Release the line and, within the resulting connection dialog, establish an Action method on the Touch Up Inside or Touch Down event configured to call a method named **saveData**. Repeat this step to create an action connection from the Find button to a method named **findContact**.

On completion of these steps, the ViewController.h file should read as follows:

```
@property (weak, nonatomic) IBOutlet UITextField *theName;  
@property (weak, nonatomic) IBOutlet UITextField *theAddress;  
@property (weak, nonatomic) IBOutlet UITextField *thePhone;  
@property (weak, nonatomic) IBOutlet UILabel *theStatus;
```

```
- (IBAction)saveData:(UIButton *)sender;  
- (IBAction)findContact:(UIButton *)sender;
```

DON'T synthesize the properties inside of ViewController.m. Instead we are going to take advantage of an auto-synthesize feature of the latest iOS SDK. We get auto synthesize if we use the names above plus the underscore "\_" in front of the name. For example:

```
theName is synthesized as __theName  
theAddress is synthesized as __theAddress  
thePhone is synthesized as __thePhone  
theStatus is synthesized as __theStatus
```

We also get **\_\_databasePath** and **\_\_contractDB** automatically as well.

## 6. Creating the Database and Table

When the application is launched it will need to check whether the database file already exists and, if not, create both the database file and a table within the database in which to store the contact

information entered by the user. The code to perform this task can be placed in the **viewDidLoad** method of our view controller class.

Select the **ViewController.m** file, scroll down to the viewDidLoad method and modify it as follows:

```
- (void)viewDidLoad
{
    [super viewDidLoad];
    NSString *docsDir;
    NSArray *dirPaths;

    // Get the documents directory
    dirPaths = NSSearchPathForDirectoriesInDomains(
        NSDocumentDirectory, NSUserDomainMask, YES);

    docsDir = dirPaths[0];

    // Build the path to the database file
    _databasePath = [[NSString alloc]
        initWithString: [docsDir
            stringByAppendingPathComponent:@"contacts.db"]];

    NSFileManager *filemgr = [NSFileManager defaultManager];

    if ([filemgr fileExistsAtPath: _databasePath] == NO)
    {
        const char *dbpath = [_databasePath UTF8String];

        if (sqlite3_open(dbpath, &_contactDB) == SQLITE_OK)
        {
            char *errMsg;
            const char *sql_stmt =
                "CREATE TABLE IF NOT EXISTS CONTACTS (ID INTEGER
                PRIMARY KEY AUTOINCREMENT, NAME TEXT, ADDRESS TEXT,
                PHONE TEXT)";

            if (sqlite3_exec(_contactDB, sql_stmt, NULL, NULL, &errMsg)
                != SQLITE_OK)
            {

```

```

        _theStatus.text = @"Failed to create table";
    }
    sqlite3_close(_contactDB);
} else {
    _theStatus.text = @"Failed to open/create database";
}
}
}

```

The code in the above method performs the following tasks:

- Identifies the application's Documents directory and constructs a path to the contacts.db database file.
- Creates an NSFileManager instance and subsequently uses it to detect if the database file already exists.
- If the file does not yet exist the code converts the path to a UTF-8 string and creates the database via a call to the SQLite `sqlite3_open()` function, passing through a reference to the `contactDB` variable declared previously in the interface file.
- Prepares a SQL statement to create the contacts table in the database.
- Reports the success or otherwise of the operation via the status label.
- Closes the database.

## 7. Implementing the Code to Save Data to the SQLite Database

The saving of contact data to the database is the responsibility of the **saveData** action method. This method will need to open the database file, extract the text from the three text fields and construct and execute a SQL INSERT statement to add this data as a record to the database. Having done this, the method will then need to close the database.

In addition, the code will need to clear the text fields ready for the next contact to be entered, and update the status label to reflect the success or failure of the operation.

In order to implement this behavior, we need to modify the **saveData** method as follows:

```

- (IBAction)saveData:(UIButton *)sender {
    sqlite3_stmt *statement;

```

```

const char *dbpath = [_databasePath UTF8String];

if (sqlite3_open(dbpath, &_contactDB) == SQLITE_OK)
{
    NSString *insertSQL = [NSString stringWithFormat:
        @"INSERT INTO CONTACTS (name, address, phone) VALUES
        (\"%@\", \"%@\", \"%@\"), self.theName.text, self.theAddress.text,
        self.thePhone.text];

    const char *insert_stmt = [insertSQL UTF8String];
    sqlite3_prepare_v2(_contactDB, insert_stmt,
        -1, &statement, NULL);
    if (sqlite3_step(statement) == SQLITE_DONE)
    {
        self.theStatus.text = @"Contact added";
        self.theName.text = @"";
        self.theAddress.text = @"";
        self.thePhone.text = @"";
    } else {
        self.theStatus.text = @"Failed to add contact";
    }
    sqlite3_finalize(statement);
    sqlite3_close(_contactDB);
}
}

```

## 8. Implementing the Code to Find Data in the SQLite Database

The next step is to implement the action for the **findContact** button. Implementing Code to Extract Data from the SQLite Database

As previously indicated, the user will be able to extract the address and phone number for a contact by entering the name and touching the find button. To this end, the Touch Up Inside event of the find button has been connected to the **findContact** method, the code for which is outlined below:

```

- (IBAction)findContact:(UIButton *)sender {

    const char *dbpath = [_databasePath UTF8String];

```

```

sqlite3_stmt *statement;

if (sqlite3_open(dbpath, &_contactDB) == SQLITE_OK)
{
    NSString *querySQL = [NSString stringWithFormat:
        @"SELECT address, phone FROM contacts WHERE
name=\"%@\"", _theName.text];

    const char *query_stmt = [querySQL UTF8String];

    if (sqlite3_prepare_v2(_contactDB,
        query_stmt, -1, &statement, NULL) == SQLITE_OK)
    {
        if (sqlite3_step(statement) == SQLITE_ROW)
        {
            NSString *addressField = [[NSString alloc]
                initWithUTF8String:(const char *)
sqlite3_column_text(statement, 0)];
            _theAddress.text = addressField;
            NSString *phoneField = [[NSString alloc]
                initWithUTF8String:(const char *)
                sqlite3_column_text(statement, 1)];
            _thePhone.text = phoneField;
            _theStatus.text = @"Match found";
        } else {
            _theStatus.text = @"Match not found";
            _theAddress.text = @"";
            _thePhone.text = @"";
        }
        sqlite3_finalize(statement);
    }
    sqlite3_close(_contactDB);
}
}

```

Here is what the above source code is doing:

- This code opens the database and constructs a SQL SELECT statement to extract any records in the database that match the name entered by the user into the name text field.



- The SQL statement is then executed. A return value of SQLITE\_ROW indicates that at least one match has been located. In this case the first matching result data is extracted and assigned to NSString objects and displayed in the appropriate text fields.
- As an alternative, a while loop could have been constructed to display all matching results. For the purposes of keeping this example simple, however, we will display only the first match.
- The code then updates the status label to indicate whether a match was found and closes the database.

9. Optional: Make the keyboard go away when user finishes entering text. Otherwise, the keyboard might cover up the buttons to save and search.

Wire each of the textField, "Did End on Exit" events to your ViewController.h file as follows:

- (IBAction)NameDone:(UITextField \*)sender;
- (IBAction)addressDone:(UITextField \*)sender;
- (IBAction)phoneDone:(UITextField \*)sender;

Add the following code to resign the first responded in your ViewController.m implementation of these methods:

- (IBAction)NameDone:(UITextField \*)sender {  
    [\_theName resignFirstResponder];  
}
- (IBAction)addressDone:(UITextField \*)sender {  
    [\_theAddress resignFirstResponder];  
}
- (IBAction)phoneDone:(UITextField \*)sender {  
    [\_thePhone resignFirstResponder];  
}

10. Building and Running the Application

Enter details for a few contacts, pressing the Save button after each entry. Be sure to check the status label to ensure the data is being saved successfully. Finally, enter the name of one of your contacts and click on the Find button. Assuming the name matches a previously entered record, the address and phone number for that contact should be displayed and the status label updated with the message "Match found."

Now, go use data in your other applications!